

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**9. (Node2vec)** Consider an undirected graph  $G$  given in the following figure. A random walk has just moved from node  $A$  to node  $B$ , with  $p = 2$  and  $q = 0.5$ .

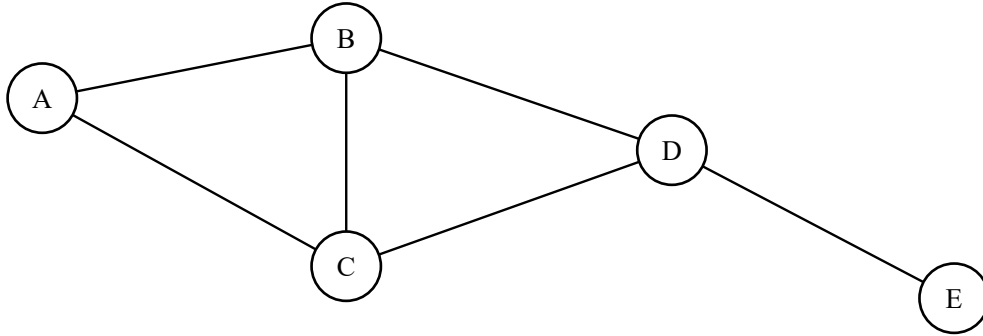


Figure 1. Graph G

The unnormalized transition probability from  $B$  to a next node  $x$  is  $\pi_{Bx} = \alpha_{pq}(A, x)$ , where

$$\alpha_{pq}(A, x) = 1/p \text{ if } d(A, x) = 0; \quad 1 \text{ if } d(A, x) = 1; \quad 1/q \text{ if } d(A, x) = 2$$

and  $d(A, x)$  is the shortest path distance between node  $A$  and node  $x$ .

- a) Explain BFS and DFS, and explain which one a small  $q$  makes the walk behave like.
- b) List all possible next nodes from node  $B$  and calculate the unnormalized transition probabilities  $\pi_{Bx}$ .
- c) Calculate the normalized transition probabilities.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**10. (LINE)** Consider an undirected graph  $G$  with node embeddings

$$u_A = 1, u_B = 0.5, u_C = -0.5, u_D = -1$$

and context embeddings

$$u'_A = 0.5, u'_B = 1, u'_C = -1, u'_D = -0.5.$$

The second-order proximity is defined as

$$p_2(j|x) = \exp(u'_j u_x) / \sum_k \exp(u'_k u_x).$$

Given  $e^{0.25} = 1.28$ ,  $e^{0.5} = 1.65$ ,  $e^{-0.25} = 0.78$ ,  $e^{-0.5} = 0.61$ .

- a) What is second-order proximity, and how does it differ from first-order proximity?
- b) Calculate  $p_2(A|B)$ ,  $p_2(B|B)$ ,  $p_2(C|B)$  and  $p_2(D|B)$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**11. (GCN)** Given a graph with an adjacency matrix  $A$  and initial node feature matrix  $H^{(0)}$  as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ -1 \\ 2 \\ 0 \\ 3 \end{bmatrix}$$

Assume that the hidden layer of a GCN model of all nodes at layer  $(k)$  can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}),$$

where  $H^{(k)}$  denotes the output at layer  $k$ ,  $\tilde{A}$  is the normalized matrix ( $\tilde{A} = D^{-1}A$ ), and  $\sigma$  is a ReLU function  $\text{ReLU}(x) = \max(0, x)$ .

- a) Explain how information is propagated in a GCN layer.
- b) Calculate  $\tilde{A}$ .
- c) Calculate the output of the GCN model at layer  $k = 1$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**12. (GraphSAGE)** Consider an undirected graph  $G$  of five nodes A, B, C, D and E given in the following figure. Each node has initial features that are the numbers standing next to it (i.e. the initial feature of node 'A' is  $h_A^{(0)} = 0.4$ ). According to the GraphSAGE model with an AGGREGATE that is a MEAN function, the feature of a node  $i$  at layer  $k$  can be updated as:

$$h_{N(i)}^{(k)} = \text{AGGREGATE}(\{ h_u^{(k-1)}, \forall u \in N(i) \})$$

$$h_i^{(k)} = \text{ReLU}( h_i^{(k-1)} \parallel h_{N(i)}^{(k)} )$$

where  $\parallel$  is a concatenation,  $\text{ReLU}(x) = \max(0, x)$ , and  $N(i)$  is the set of neighbour nodes of node  $i$ .

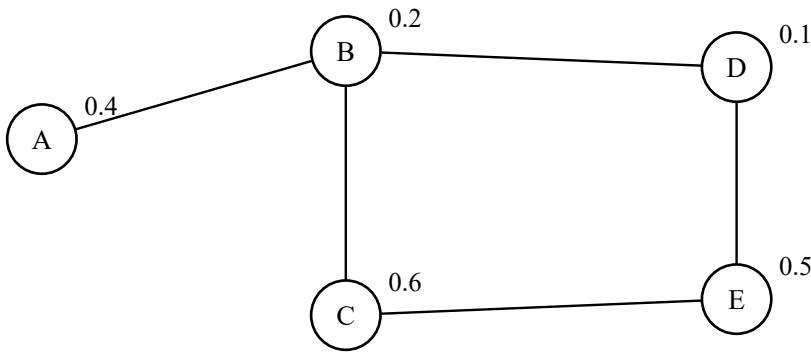


Figure 2. Graph G

- What is the main idea behind the GraphSAGE model, and how does it differ from traditional GCNs?
- Calculate the feature of each node at  $k = 1$ .
- Calculate a graph-level embedding  $h_G$  by using a 'Mean' global pooling when  $k = 1$ .

### Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**13. (ClusterGCN)** Consider an undirected graph  $G$  of six nodes A, B, C, D, E and F given in the following figure. The graph  $G$  contains two clusters  $G_1$  and  $G_2$ . Each node has initial features that are the numbers standing next to it. According to the ClusterGCN model, the feature of a node  $i$  at layer  $k$  can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}( h_i^{(k-1)} \parallel h_{N(i)}^{(k)} )$$

where  $\parallel$  is a concatenation and  $\text{ReLU}(x) = \max(0, x)$ .

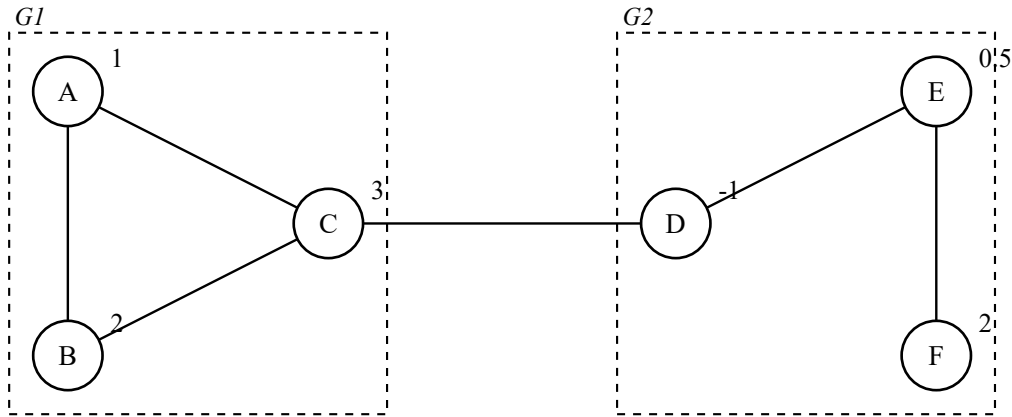


Figure 3. Graph G with two clusters

- What is mini-batch training, and why does ClusterGCN need it?
- Calculate the output representations of all nodes at layer  $k = 1$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**14. (GraphSAINT)** Consider an undirected graph  $G$  of six nodes A, B, C, D, E and F given in the following figure. The graph  $G$  has a sampled subgraph  $G_1$ . Each node has initial features that are the numbers standing next to it. According to the GraphSAINT model, the feature of a node  $i$  at layer  $k$  can be updated as:

$$h_{N(i)}^{(k)} = \text{MEAN}(\{ h_u^{(k-1)}, \forall u \in N(i), G_u = G_i \})$$

$$h_i^{(k)} = \text{ReLU}(h_i^{(k-1)} \parallel h_{N(i)}^{(k)})$$

where  $\parallel$  is a concatenation and  $\text{ReLU}(x) = \max(0, x)$ .

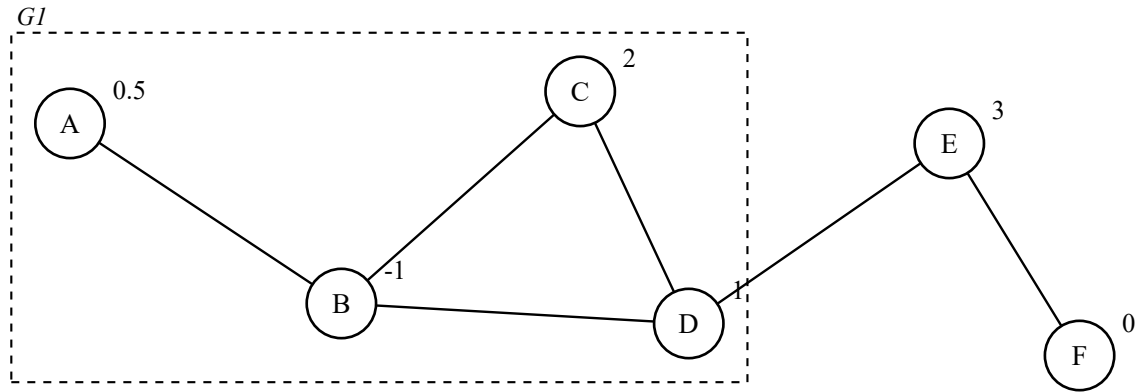


Figure 4. Graph  $G$  with a sampled subgraph  $G_1$

- a) How does GraphSAINT's sampling strategy differ from node-wise or layer-wise sampling methods like GraphSAGE and FastGCN?
- b) Calculate the output representations of nodes A, B, C and D at layer  $k = 1$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**15. (JK Network)** Given a graph with an adjacency matrix  $A$  and initial node feature matrix  $H^{(0)}$  as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} -1 \\ 0 \\ 1 \\ -1 \\ 2 \end{bmatrix}$$

Assume that the output of a JK network model of all nodes at layer  $(k)$  can be calculated as:

$$H^{(k)} = \max( \sigma(\tilde{A} \cdot H^{(0)}), \sigma(\tilde{A} \cdot H^{(1)}), \dots, \sigma(\tilde{A} \cdot H^{(k-1)}) )$$

where  $H^{(k)}$  denotes the output at layer  $k$ ,  $\tilde{A}$  is the normalized matrix ( $\tilde{A} = D^{-1}A$ ), and  $\sigma$  is a ReLU function  $\text{ReLU}(x) = \max(0, x)$ .

- a) Explain how Jumping Knowledge Networks help mitigate the over-smoothing problem in deep GCNs.
- b) Calculate  $\tilde{A}$ .
- c) Calculate the output representations at layer  $k = 2$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**16. (GCNII)** Given a graph with an adjacency matrix  $A$  and initial node feature matrix  $H^{(0)}$  as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 3 \\ -1 \\ 1 \\ 2 \\ -3 \end{bmatrix}$$

Assume that the output of a GCNII model of all nodes at layer  $(k)$  can be calculated as:

$$H^{(k)} = \sigma[ ((1 - \beta) I_n) \cdot ((1 - \alpha) \tilde{A} \cdot H^{(k-1)} + \alpha H^{(0)}) ]$$

where  $H^{(k)}$  denotes the output at layer  $k$ ,  $\tilde{A}$  is the normalized matrix ( $\tilde{A} = D^{-1}A$ ),  $I_n$  is the identity matrix,  $\alpha = \beta = 0.5$ , and  $\sigma$  is a ReLU function  $\text{ReLU}(x) = \max(0, x)$ .

- a) What are the two major components introduced in GCNII to overcome the depth limitation problem in GCNs?
- b) Calculate  $\tilde{A}$ .
- c) Calculate the output representations at layer  $k = 1$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**17. (DeepGCN)** Given a graph with an adjacency matrix  $A$  and initial node feature matrix  $H^{(0)}$  as follows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ -2 \\ 1 \\ 0 \end{bmatrix}$$

Assume that the output of a DeepGCN model with a residual connection of all nodes at layer  $(k)$  can be calculated as:

$$H^{(k)} = \sigma(\tilde{A} \cdot H^{(k-1)}) + H^{(k-1)}$$

where  $\tilde{A} = D^{-1}A$  and  $\sigma$  is a ReLU function  $\text{ReLU}(x) = \max(0, x)$ .

- a) Explain why a residual connection allows a GCN to be made deeper.
- b) Calculate the output representations at layer  $k = 1$  and  $k = 2$ .
- c) Compare  $H^{(1)}$  with the output of a plain GCN without the residual connection.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**1. (Relational Classification)** Consider an undirected graph  $G$  of five nodes given in the following figure. Nodes 1 and 5 are labelled ( $P(Y_1) = 1, P(Y_5) = 0$ ) and the others are unlabelled and initialized to 0.5. The relational classifier updates every unlabelled node as:

$$P(Y_v = c) = ( \sum_{(v,u) \in E} A_{v,u} P(Y_u = c) ) / ( \sum_{(v,u) \in E} A_{v,u} )$$

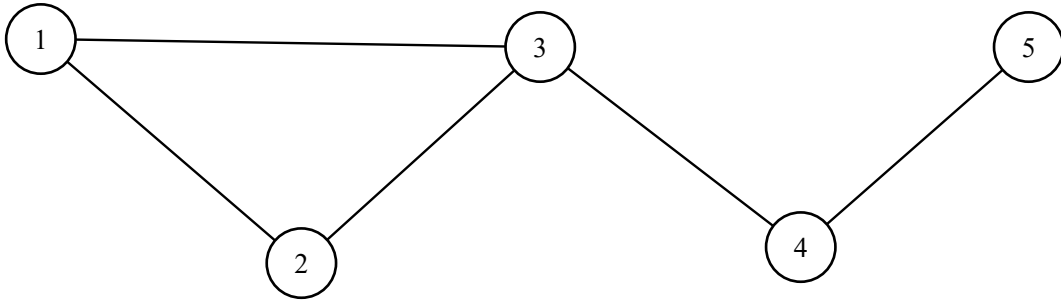


Figure 5. Graph G

- a) Explain the key assumption behind relational classification, and state one limitation of it.
- b) Calculate  $P(Y_v)$  for every unlabelled node after the 1st iteration.
- c) Calculate  $P(Y_v)$  after the 2nd iteration. Update nodes in the order 2, 3, 4 using the values from the previous iteration only.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**2. (FastGCN)** Consider the undirected graph  $G$  with adjacency matrix  $A$  and node feature vector  $h^{(0)}$  below. FastGCN replaces the full neighbourhood sum of a GCN layer by a layer-wise Monte-Carlo estimate:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$h^{(0)} = \begin{bmatrix} 1 \\ 2 \\ -1 \\ 0 \\ 3 \end{bmatrix}$$

$$h^{(l+1)}(v) = \sigma \left( \frac{1}{t_l} \sum_{j=1}^{t_l} \tilde{A}(v, u_j) h^{(l)}(u_j) \right)$$

where  $\tilde{A} = D^{-1}A$ , the  $t_l$  nodes  $u_j$  are drawn once per layer from the importance distribution  $q(u) \propto \|\tilde{A}(:, u)\|^2$ , and  $\text{ReLU}(x) = \max(0, x)$ .

- a) How does layer-wise sampling in FastGCN differ from the node-wise sampling of GraphSAGE, and what problem does it solve?
- b) Calculate the importance distribution  $q(u)$  over the five nodes.
- c) Suppose the sampled set for the layer is  $\{B, C\}$  ( $t_l = 2$ ). Calculate the estimate of  $h^{(1)}(A)$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**3. (ASGCN)** Consider the same graph  $G$  as in question 11, with  $\tilde{A} = D^{-1}A$  and the node feature vector  $h^{(0)}$  below. ASGCN rewrites a GCN layer as an expectation:

$$h^{(0)} = \begin{bmatrix} 1 \\ 2 \\ -1 \\ 0 \\ 3 \end{bmatrix}$$

$$h^{(l+1)}(v_i) = \sigma( W^{(l)} N(v_i) E_{p(u_j | v_i)}[ h^{(l)}(u_j) ] )$$

where  $N(v_i) = \sum_j \tilde{A}_{i,j}$  is the normalization factor and  $p(u_j | v_i) = \tilde{A}_{i,j} / N(v_i)$  is the probability of sampling node  $u_j$  given the parent node  $v_i$ . Take  $W^{(l)} = 1$  and  $\text{ReLU}(x) = \max(0, x)$ .

- a) ASGCN is called 'adaptive' sampling. What does the proposal distribution depend on in ASGCN that it does not depend on in FastGCN?
- b) Calculate  $N(v_i)$  and  $p(u_j | v_i)$  for the parent node  $v_i = B$ .
- c) Calculate  $E_{p(u_j | B)}[h^{(0)}(u_j)]$  and the resulting  $h^{(1)}(B)$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**4. (APPNP)** Given a graph with an adjacency matrix  $A$  and a prediction matrix  $H$  produced by an MLP as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

APPNP (Approximate Personalized Propagation of Neural Predictions) propagates the predictions with a teleport probability  $\alpha$ :

$$Z^{(0)} = H, \quad Z^{(k+1)} = (1 - \alpha) \tilde{A} Z^{(k)} + \alpha H$$

where  $\tilde{A} = D^{-1}A$  and  $\alpha = 0.5$ . No nonlinearity is applied between propagation steps.

- a) Explain the teleport mechanism of Personalized PageRank and why it prevents over-smoothing even with many propagation steps.
- b) Calculate  $\tilde{A}$  and  $Z^{(1)}$ .
- c) Calculate  $Z^{(2)}$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**5. (GPR-GNN)** Given a graph with an adjacency matrix  $A$  and hidden features  $H^{(0)}$  produced by a neural network as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 0 \\ 1 \\ -1 \end{bmatrix}$$

GPR-GNN (Generalized PageRank GNN) separates feature extraction from propagation:

$$H^{(k)} = \tilde{A} H^{(k-1)}, \quad Z = \sum_{k=0}^K \gamma_k H^{(k)}$$

where  $\tilde{A} = D^{-1}A$ , the weights  $\gamma_k$  are learned, and here  $K = 2$  with  $\gamma_0 = 0.5$ ,  $\gamma_1 = 0.3$ ,  $\gamma_2 = -0.2$ .

- a) What can GPR-GNN do that APPNP cannot, and why does allowing negative  $\gamma_k$  matter for heterophilic graphs?
- b) Calculate  $H^{(1)}$  and  $H^{(2)}$ .
- c) Calculate  $Z$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**6. (MixHop)** Given a graph with an adjacency matrix  $A$  and initial node feature matrix  $H^{(0)}$  as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}$$

A MixHop layer concatenates several powers of the adjacency matrix:

$$H^{(i+1)} = \mathbb{I}_{j \in P} \sigma(\hat{A}^j H^{(i)} W_j^{(i)})$$

where  $\mathbb{I}$  is a column-wise concatenation,  $P = \{0, 1, 2\}$ ,  $\hat{A} = D^{-1}A$ ,  $W_j^{(i)} = 1$  for every  $j$ , and  $\text{ReLU}(x) = \max(0, x)$ .

- a) State Theorem 1 of MixHop about the two-hop Delta Operator, and explain why a vanilla GCN cannot represent it even with infinitely many layers.
- b) Calculate  $\hat{A}^0 H^{(0)}$ ,  $\hat{A}^1 H^{(0)}$  and  $\hat{A}^2 H^{(0)}$ .
- c) Calculate  $H^{(1)}$ , and calculate the Delta Operator  $\sigma(\hat{A}^0 H^{(0)}) - \sigma(\hat{A}^2 H^{(0)})$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**7. (PairNorm)** After one GCN layer the node features of a four-node graph are

$$X = \begin{bmatrix} 3 & 1 \\ 1 & -1 \\ -1 & 2 \\ 1 & 2 \end{bmatrix}$$

PairNorm normalizes them in two steps, a centering step and a scaling step:

$$x_i^c = x_i - (1/n) \sum_{i=1}^n x_i \quad (\text{Center})$$

$$x_i = s \cdot x_i^c / \sqrt{(1/n) \sum_{i=1}^n \|x_i^c\|_2^2} \quad (\text{Scale})$$

with  $s = 1$  and  $n = 4$ .

- a) Explain node-wise and feature-wise over-smoothing, and explain which quantity PairNorm keeps constant across layers.
- b) Calculate the centered features  $x_i^c$ .
- c) Calculate the scaling denominator and the final  $x_i$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**8. (DropEdge)** Consider the complete graph  $K_4$  on nodes A, B, C and D with the adjacency matrix  $A$  and initial node feature matrix  $H^{(0)}$  below.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$H^{(0)} = \begin{bmatrix} 2 \\ 4 \\ -2 \\ 0 \end{bmatrix}$$

DropEdge keeps each undirected edge independently with probability  $1 - p$  at every training epoch, and the layer is then computed on the surviving graph:

$$H^{(1)} = \sigma( (D^{(t)})^{-1} A^{(t)} H^{(0)} )$$

where  $A^{(t)}$  and  $D^{(t)}$  are rebuilt from the surviving edges only,  $p = 1/3$ , and  $\text{ReLU}(x) = \max(0, x)$ .

- a) Which two problems does DropEdge target, and how does it differ from Dropout?
- b) Calculate the expected number of surviving edges, and calculate  $H^{(1)}$  on the full graph.
- c) Suppose the edges (A, D) and (B, C) are dropped in this epoch. Recalculate  $H^{(1)}$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**18. (GAT)** Consider an undirected graph  $G$  of four nodes A, B, C and D. Each node has an initial feature next to it ( $h_A^{(0)} = 0.3, h_B^{(0)} = 0.1, h_C^{(0)} = 0.2, h_D^{(0)} = 0.2$ ). The weight matrix is  $W = [0.5]$ .

$$h_i^{(k)} = \sigma(\sum \alpha_{im} W h_m), \quad m \in N(i)$$

$$\alpha_{im} = e_{im} / \sum e_{ik}, \quad e_{im} = \sigma(\text{MEAN}(W h_i, W h_m))$$

where  $\sigma$  is a ReLU function  $\text{ReLU}(x) = \max(0, x)$ . Node A is connected to B, C and D.

- a) What problem in traditional GCNs does GAT aim to solve?
- b) Calculate the attention coefficients  $e_{AB}$ ,  $e_{AC}$  and  $e_{AD}$ .
- c) Calculate the feature of node A at  $k = 1$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**19. (GATv2)** Compare the attention score  $e_{ij}$  of GAT and GATv2.

$$GAT: \quad e_{ij} = \text{LeakyReLU}(a(W h_i, W h_j))$$

$$GATv2: \quad e_{ij} = a \text{ LeakyReLU}(W[h_i, h_j])$$

$\text{vec } h_i = [1, 2]^T$ ,  $\text{vec } h_j = [1, -1]^T$ ,  $W = I_2$ ,  $a = [1, 1, 1, 1]$ , and  $\text{LeakyReLU}(x) = x$  if  $x > 0$ ,  $0.02x$  if  $x < 0$ .

- a) What is the key limitation of GAT that GATv2 aims to fix?
- b) Calculate  $e_{ij}$  for both and compare.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**20. (HAN)** Consider a heterogeneous academic graph with three node types: Author (A), Paper (P) and Venue (V). Initial features are  $h_{P1} = 0.3$ ,  $h_{P2} = 0.1$ ,  $h_{P3} = 0.2$ ,  $h_{A1} = 0.1$ ,  $h_{A2} = 0.1$ ,  $h_{V1} = 0.2$ , and  $W = [0.5]$ . Edges:

$P1 - A1$ ,  $P2 - A1$ ,  $P2 - A2$ ,  $P3 - A2$ ,  $P1 - V1$ ,  $P3 - V1$ .

$$h_i^{(k)} = \sigma(\sum \alpha_{im} W h_m), \quad m \in N(i)$$

$$\alpha_{im} = e_{im} / \sum e_{ik}, \quad e_{im} = \sigma(\text{MEAN}(W h_i, W h_m))$$

The superscript  $\Phi$  means the meta-path.  $\sigma$  is a ReLU function.

- a) What is a heterogeneous graph?
- b) List all meta-paths PAP and PVP, and calculate the attention coefficients.
- c) Calculate the feature of node  $P1$  at  $k = 1$ .

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**21. (Homophily Metrics)** Consider the undirected graph  $G$  below. The number next to each node is its class label  $y$ .

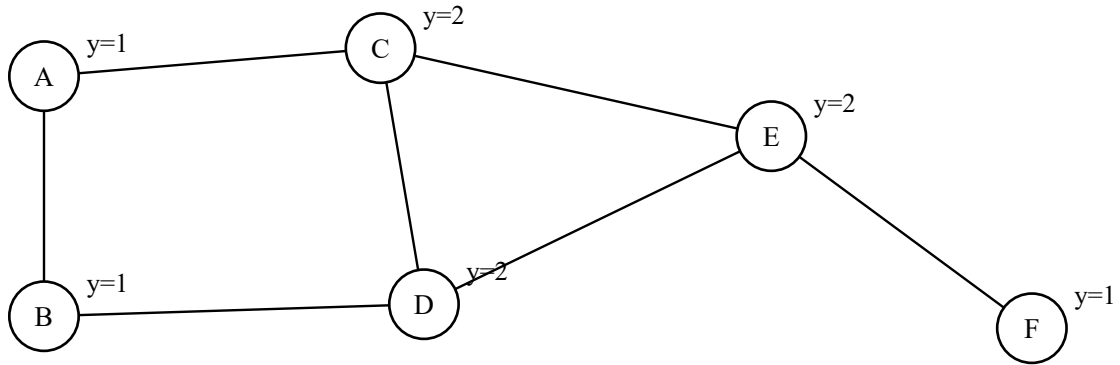


Figure 1. A graph with two classes

Four homophily metrics are defined as follows ( $C$  is the number of classes):

$$h_{edge} = |\{(u,v) \text{ in } E : y_u = y_v\}| / |E|$$

$$h_{node} = (1/|V|) \sum_v |\{u \text{ in } N(v) : y_u = y_v\}| / d(v)$$

$$h_{class} = (1/C) \sum_k \max(0, h_k - |V_k|/|V|), \quad h_k = |\{(u,v) \text{ in } E : y_u = y_v = k\}| / |\{(u,v) \text{ in } E : y_u = k\}|$$

$$h_{adj} = (h_{edge} - \sum_k p_k^2) / (1 - \sum_k p_k^2), \quad p_k = \sum_{v \text{ in } V_k} d(v) / 2|E|$$

- Explain why edge homophily can be misleading, and explain what node homophily and adjusted homophily each correct.
- Calculate  $h_{edge}$  and  $h_{node}$ .
- Calculate  $h_{class}$  and  $h_{adj}$ , and state whether this graph is homophilic.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**22. (Label Informativeness)** Consider the two graphs below. Both have four nodes ( $y_A = y_B = 1, y_C = y_D = 2$ ) and four edges.



Figure 1. Graph G1 (left: A, B, C, D) and Graph G2 (right: E, F, G, H)

Label informativeness measures the mutual information between the labels at the two ends of an edge:

$$LI = I(Y_u ; Y_v) / H(Y_u)$$

$$I(Y_u; Y_v) = \sum_{p,q} P(p,q) \log_2 ( P(p,q) / (P(p) P(q)) ), \quad H(Y_u) = - \sum_p P(p) \log_2 P(p)$$

$P(p,q)$  is the probability that a randomly chosen directed edge goes from a node of class  $p$  to a node of class  $q$ .

- Explain why edge homophily alone can be a misleading measure of heterophily.
- Calculate  $h_{edge}$  and  $LI$  for graph G1.
- Calculate  $h_{edge}$  and  $LI$  for graph G2, and explain which graph a GCN can solve.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**23. (FAGCN)** Consider the path graph  $A - B - C$  with the node signal  $x = [1, 2, 3]^T$  (one feature per node).

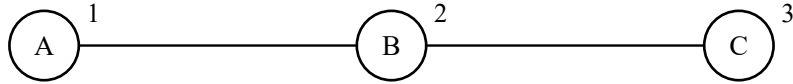


Figure 1. Path graph with signal  $x$

FAGCN defines an enhanced low-pass filter and an enhanced high-pass filter with a scaling parameter  $\epsilon \in (0, 1)$ :

$$F_L = \epsilon I + D^{-1/2} A D^{-1/2} = (\epsilon + 1) I - L$$

$$F_H = \epsilon I - D^{-1/2} A D^{-1/2} = (\epsilon - 1) I + L$$

Use  $\epsilon = 0.5$ . The self-gating coefficient of an edge is  $\alpha_{ij} = \tanh(g^T [h_i \parallel h_j])$ .

- a) Explain what low-frequency and high-frequency signals mean on a graph, and explain why a GCN cannot handle heterophily.
- b) Calculate  $D^{-1/2} A D^{-1/2}$ , then  $F_L \cdot x$  and  $F_H \cdot x$ .
- c) Given  $g^T[h_A \parallel h_B] = 0.4$  and  $g^T[h_B \parallel h_C] = -0.8$ , calculate the two gating coefficients and explain what a negative coefficient does.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**24. (H2GCN)** Consider the undirected graph  $G$  below with the embedding  $H^{(0)} = [1, 2, 3, 4, 5]^T$  (one value per node, in the order A, B, C, D, E).

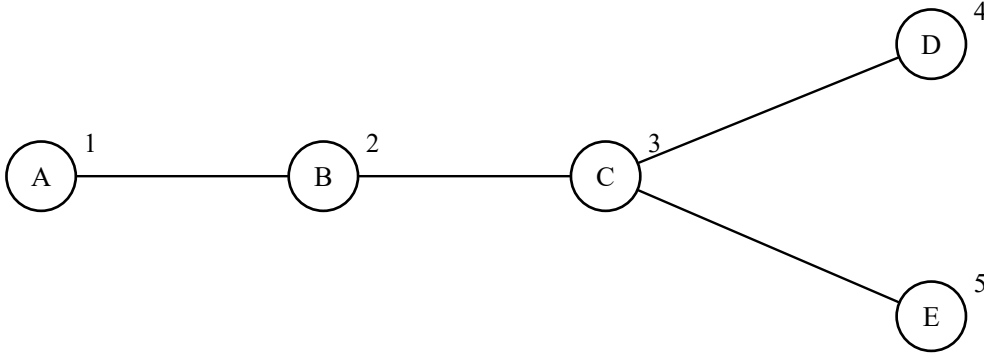


Figure 1. Graph G

H2GCN uses three designs: (D1) ego- and neighbour-embedding separation, (D2) higher-order neighbourhoods, and (D3) a combination of all intermediate representations. With MEAN aggregation and one round  $K = 1$ :

$$R_1^{(1)} = \text{MEAN}\{ h_u^{(0)} : u \text{ in } N_1(v) \}, \quad R_2^{(1)} = \text{MEAN}\{ h_u^{(0)} : u \text{ in } N_2(v) \}$$

$$H_{final} = \text{CONCAT}( H^{(0)}, R_1^{(1)}, R_2^{(1)} )$$

$N_1(v)$  is the set of 1-hop neighbours and  $N_2(v)$  is the set of nodes at distance exactly two. Neither set contains  $v$  itself.

- a) Explain the three designs D1, D2 and D3, and explain why 2-hop neighbourhoods help under heterophily.
- b) List  $N_1(v)$  and  $N_2(v)$  for every node and calculate  $R_1^{(1)}$  and  $R_2^{(1)}$ .
- c) Write the final representation  $H_{final}$  of every node.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**25. (ACM)** Consider the path graph  $A - B - C$  with  $H^{(0)} = [1, 2, 3]^T$ . The Adaptive Channel Mixing (ACM) framework builds three channels per layer.

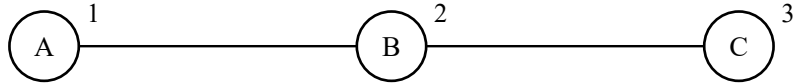


Figure 1. Path graph

With the low-pass filter  $H_{LP} = \hat{A}_{rw} = D^{-1}A$ , the high-pass filter  $H_{HP} = I - \hat{A}_{rw}$ , and all weight matrices equal to the identity:

$$H_L = \text{ReLU}(H_L P H), \quad H_H = \text{ReLU}(H_H P H), \quad H_I = \text{ReLU}(I H)$$

$$a_{\sim L} = \text{Sigmoid}(H_L), \quad a_{\sim H} = \text{Sigmoid}(H_H), \quad a_{\sim I} = \text{Sigmoid}(H_I)$$

$$(a_L, a_H, a_I) = \text{Softmax}((a_{\sim L}, a_{\sim H}, a_{\sim I}) / T), \quad T = 1$$

$$H' = \text{ReLU}(\text{diag}(a_L) H_L + \text{diag}(a_H) H_H + \text{diag}(a_I) H_I)$$

- a) Explain the role of each of the three channels, and explain why ACM learns the mixing weights per node instead of once for the whole graph.
- b) Calculate  $H_L$ ,  $H_H$  and  $H_I$ .
- c) Calculate the mixing weights of node  $A$  and its final representation.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**26. (GGCN)** Consider a star graph where node  $A$  is connected to  $B$ ,  $C$  and  $D$ . The node features are

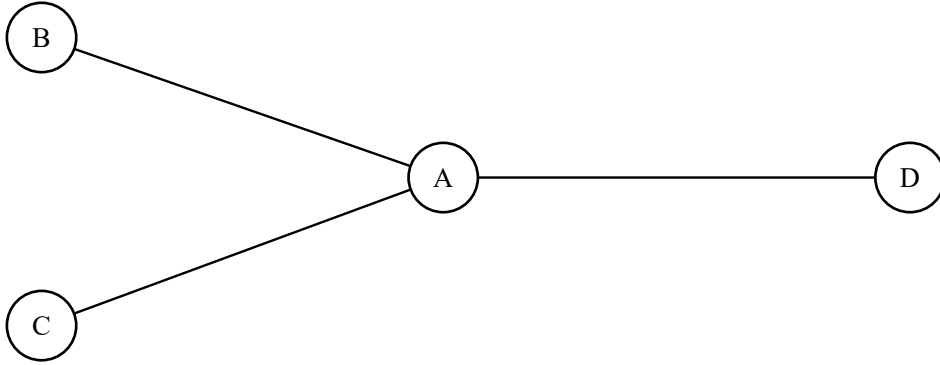


Figure 1. Star graph centred at A

$x_A = [1, 0]^T$ ,  $x_B = [0.8, 0.6]^T$ ,  $x_C = [-1, 0]^T$ ,  $x_D = [0, 1]^T$ . All four vectors have unit length.

GGCN replaces the non-negative GCN weight with a signed, feature-based weight:

$$A_{ij}^{new} = \text{Similarity}(x_i, x_j) = \cos(x_i, x_j) \quad (\text{may be negative})$$

$$\text{GCN: } h_A' = \text{MEAN}(x_A, x_B, x_C, x_D)$$

$$\text{GGCN: } h_A' = x_A + \text{SUM}_{j \in N(A)} A_{Aj}^{new} x_j$$

- a) Explain why a GCN can only use non-negative edge weights, and explain the claim that heterophily and over-smoothing are two sides of the same coin.
- b) Calculate the three signed edge weights.
- c) Calculate  $h_A'$  under GCN and under GGCN, and compare the two results.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**27. (DHGR)** Consider the bipartite graph below with node features  $x_A = x_B = [1, 0]^T$  and  $x_C = x_D = [0, 1]^T$ . The labels are  $y_A = y_B = 1$  and  $y_C = y_D = 2$  (not available to the model).

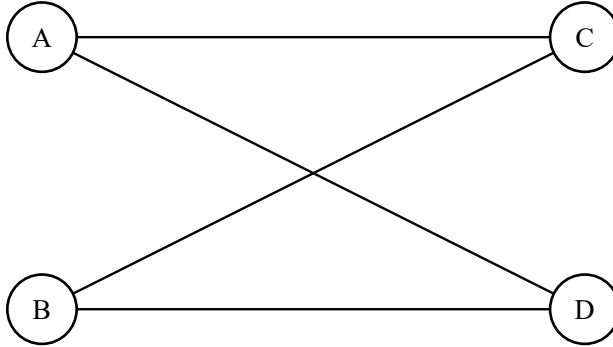


Figure 1. A fully heterophilic graph

DHGR rewires the graph before any GNN is applied. It compares the Neighborhood Feature Distribution of the nodes instead of their raw features:

$$NFD(v_i) = (1/|N(v_i)|) \sum_{v_j \in N(v_i)} x_j, \quad NFD = D^{-1} A X$$

$$S_{ij} = \alpha * \text{sim}(NFD(v_i), NFD(v_j)) + \beta * \text{sim}(NLD(v_i), NLD(v_j))$$

Use  $\alpha = 1$ ,  $\beta = 0$  and cosine similarity. Add every non-adjacent pair with  $S_{ij} = 1$  and prune every existing edge with  $S_{ij} = 0$ .

- a) Explain the difference between model-centric and data-centric approaches to heterophily, and explain why raw features cannot predict edge polarity.
- b) Calculate  $h_{edge}$  of the original graph and the NFD of every node.
- c) Calculate  $S_{ij}$  for all six pairs, perform the rewiring, and calculate the  $h_{edge}$  of the rewired graph.

## Mid-term Exam (Graph Neural Networks - Fall 2026)

Full Name:

Student ID:

Note: Students should write in English for evaluation.

**28. (SignGT)** A graph transformer computes attention from node  $i$  over three nodes  $A, B$  and  $C$ . The query, keys and values are

$q_i = [1, 0]$ ;  $k_A = [1, 0]$ ,  $k_B = [0.5, 0]$ ,  $k_C = [-1, 0]$ ;  $v_A = [1, 1]$ ,  $v_B = [2, 0]$ ,  $v_C = [0, 2]$ .

Standard self-attention and the signed self-attention (SignSA) of SignGT are

$$\text{Standard: } M_{ij} = \exp(q_i k_j^T) / \sum_k \exp(q_i k_k^T)$$

$$\text{SignSA: } M_{ij}^S = \text{sgn}(q_i k_j^T) * \exp(|q_i k_j^T|) / \sum_k \exp(|q_i k_k^T|)$$

The output is  $o_i = \sum_j M_{ij} v_j$  in both cases. Do not scale by  $\sqrt{d_k}$ .

- a) Explain why a standard graph transformer can only perform smoothing, and explain how SignSA keeps high-frequency information.
- b) Calculate the standard attention weights and the output  $o_i$ .
- c) Calculate the SignSA weights and the output  $o_i$ , and compare the two results.